National Taiwan University
Department of Electrical Engineering
Name: Bo-Yu Lien
ID: b13901072

115-1 (Fall 2026)
Reinforcement Learning

Assignment 3 Report

1 2048

Q1. Monte Carlo evaluation of $Q^\pi(s,a)$ (3 pts.)

What I estimate

- The policy is greedy: $\pi(s) = \arg\max_a Q(s,a)$. I do not replace an illegal action with another action.
- I collect 12 states from this policy's own games, using seeds starting from 71000. I take states at steps 0, 5, 15 and 30, when available. A state includes the board and the consecutive illegal-move count.
- For each state, I fix the first action to $a = \pi(s)$. Then I follow the same greedy policy until the episode ends. I repeat it 64 times with independent future random seeds.

$$G_j = \sum_{t=0}^{T_j-1} \gamma^t r_t, \qquad \widehat{Q}^\pi(s,a) = \frac{1}{64}\sum_{j=1}^{64} G_j, \qquad \gamma = 0.99.$$

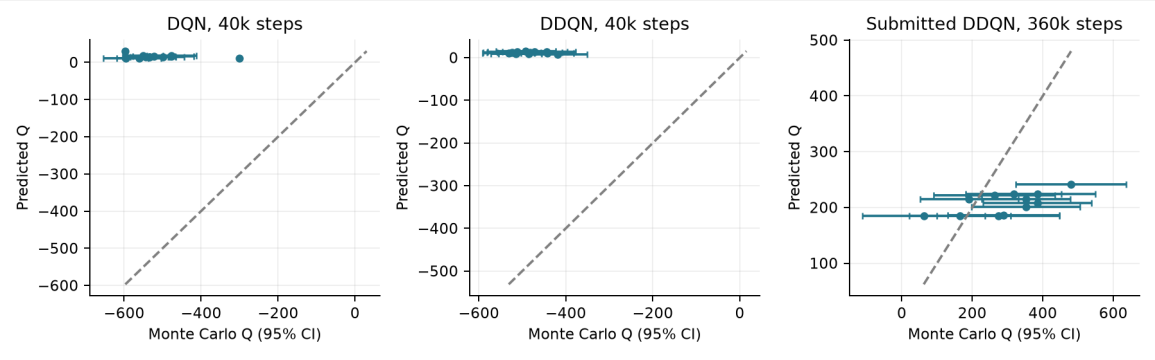
Tile randomness and confidence interval

- Every rollout starts from the same saved board, score and foul count. Only the future random-number stream changes. The environment still puts a 2 with probability 0.9 or a 4 with probability 0.1 into a random empty cell.
- The seed is $800000 + 1000i + j$, where i is the state index and j is the rollout index. The initial board is restored after resetting the random generator.
- I use a pointwise 95% Student-t interval: $\bar{G} \pm 1.99834\, s_G/\sqrt{64}$. It measures rollout randomness for one fixed state. It does not measure uncertainty across all possible boards or training runs.
- There is a 2000-step safety cap. None of the measured rollouts reached this cap, so these returns did not need bootstrapping.

Important detail

- These Q values use the submitted *training* reward and termination rule: merge score plus board shaping, illegal penalty -300, and termination after three consecutive illegal moves. This is needed to compare with the learned Q network.
- The official evaluation has no shaping and ends at the first illegal move. I measure official score separately. A training Q value is not the expected official score.
- Checkpoints do not save the old environment source. This check uses the current training environment; it cannot prove that every earlier experiment used exactly the same reward file.

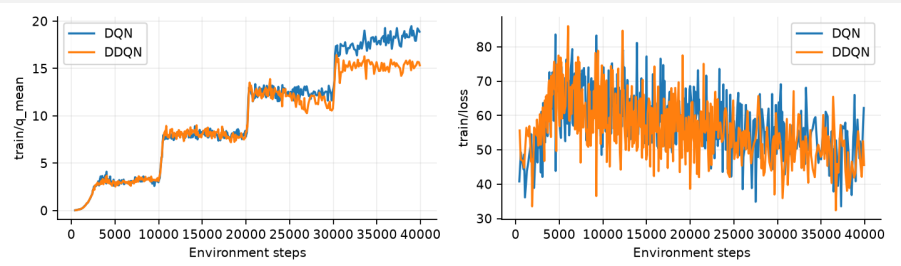
Q2. Predicted Q vs. Monte Carlo Q for DQN and DDQN (5 pts.)



The horizontal bars are 95% MC confidence intervals. The dashed line means the two values are equal. Each panel contains 12 states visited by that panel's greedy policy.

Model	Mean $Q - \widehat{Q}$	Mean absolute gap	Mean score	Best tile
DQN 40k	531.6	531.6	142.44	64
DDQN 40k	497.5	497.5	206.72	128
Submitted DDQN	-85.5	113.1	1238.00	512

- For a clean short-budget check, I train DQN and DDQN for 40,000 environment steps each. Both use seed 42, four environments, the same reward, network, replay settings and exploration schedule. Only the next-state target changes. The submitted DDQN is shown separately because it was trained longer.
- DQN uses $y = r + \gamma \max_a Q_{\text{target}}(s', a)$. The same target network selects and evaluates the action. A noisy large value can be selected by the maximum.
- DDQN first selects $a^* = \arg\max_a Q_{\text{online}}(s', a)$, then uses $y = r + \gamma Q_{\text{target}}(s', a^*)$. This separates selection from evaluation. In code, I use `gather` to get the target value at the online network's chosen index.
- Both short runs overestimate these sampled states. DDQN has a somewhat smaller mean gap in this run, but the states are different because each policy visits different boards. This small one-seed test is not strong evidence that DDQN always has lower bias.
- The final DDQN has a smaller absolute gap here, and its mean signed gap is negative. Therefore, overestimation is not the only error. More training and a different state distribution also matter.
- A small gap does not mean a good policy. A policy that stops early can correctly predict a low return. Also, shaped reward can be predicted well while the highest tile is still poor. I check actual tiles with the original evaluator.



Q means and losses from the matched 40k-step runs. The replay samples change over time, so this Q-mean plot is a training diagnostic, not a fixed-state calibration test.

Q3. Training the submitted 2048 model (4 pts.)

Final choice and network

- I submit the weights from `model_2048_weight1000.zip`, from run `ddqn_weight1000`. I rename a copy to `model_2048.zip`. The run name is a label; it does not mean the current reward has a coefficient of 1000.
- The model is Double DQN with SB3's built-in MLP policy. Input is the raw $16 \times 4 \times 4$ one-hot board. The flattened 256 inputs go through two 256-unit ReLU layers, then four Q outputs. The online network has 132,612 parameters, below the 5-million limit.

Setting	Value
Learning rate / discount	10^{-4} / 0.99
Replay size / batch size	100,000 / 256
Learning starts / target update	100 / 10,000 steps
Train frequency / gradient steps	every 4 vector steps / 1 update
Exploration	$\epsilon : 1 \to 0.05$; exploration fraction 0.1
Training budget	20 calls $\times$ 20,000 steps; four environments
Saved checkpoint	360,000 steps, selected before the final 400k step

Reward and termination

- A legal move gets its merge score plus $\Phi(B_{\text{after}}) - \Phi(B_{\text{before}})$, where $\Phi(B) = \sum_{x,y} W_{x,y} B_{x,y}$. The after-board includes the new random tile.
- The current weight matrix is below. It encourages larger tiles near the lower-left side, so the board has more order. This is ordinary reward shaping; with $\gamma < 1$, it is not a policy-invariant potential-based guarantee.

$$W = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0.5 & 0 & 0 & 0 \\ 0.75 & 0.5 & 0 & 0 \\ 1 & 0.75 & 0.5 & 0 \end{bmatrix}.$$

- An illegal move gives -300. Training allows up to three consecutive illegal moves, so exploration can try another direction. A legal move resets this counter. Official testing ends at the first illegal move, which is harder.
- After each 20k-step call, the script checks ten games and saves when mean score or mean highest tile improves. The saved file has 360k steps. These training checks were not strictly fixed-seed checks because the current reset code reseeds on reset.
- The old run did not set a training seed. The script reproduces the recipe, but exact old weights are not guaranteed. The new 40k-step comparison uses seed 42 and is not the submitted model.

Official public test, seeds 0–99

- Average score is 1238.0. Highest-tile counts are: 4:4, 8:2, 16:4, 32:10, 64:23, 128:36, 256:19, 512:2.
- At least one game reaches 128, 256 and 512, so the original evaluator gives 15/15 public baseline points. This does not guarantee the private seeds.

2 Astro Party

Q4. Pipeline for the best performance (9 pts.)

Main idea: a rule-guided policy, then imitation and PPO

- I first read the two bots and the observation layout. Rusher moves close quickly. Sniper aims from farther away. Destroying a ship is not enough: its pilot must also be killed before respawn.
- My final method is a hybrid. A deterministic defensive controller proposes turn, thrust and shoot values from the current observation. A learned SB3 actor uses this proposal and ship information. This is not an end-to-end policy that discovered every tactic by itself.
- The raw input is still the original 84 numbers. The custom extractor is defined inside the submitted training script, so it is saved with the model. No observation wrapper or normalization file is needed.

Controller logic, in execution order

1. Decode position, relative velocity, ammo, pilot status and observed bullets. Convert each coordinate with its own scale: horizontal 600 and vertical 400.
2. If my ship is a pilot, flee and survive. If the opponent is a pilot, hunt it. Otherwise, keep a useful fighting distance, roughly 500–700 pixels.
3. Aim ahead of the moving enemy, using bullet speed 620 and a velocity lead factor of 0.8. Check line of sight before deciding to fire.
4. Before the final action, simulate bullet distance for the next 60 steps. If the proposed move is unsafe, search 5 turn values $\times$ 3 thrust values. Prefer a safer move while avoiding unnecessary turns.
5. Recheck the firing direction after the chosen turn. This avoids shooting at the old heading while the ship is turning. Fire also needs ammo and cooldown readiness.

Policy and learning

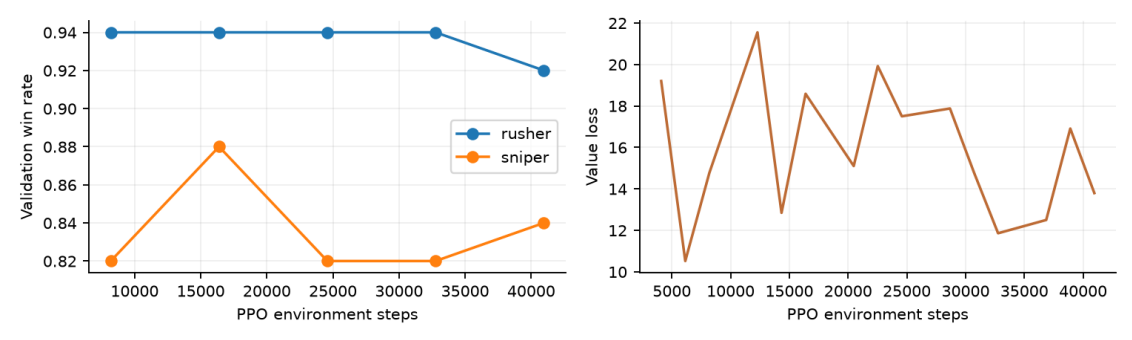
- The extractor returns the 3 controller outputs plus the first 26 observation entries: 29 features. The actor is a learned linear layer with three outputs. The value network has two 64-unit Tanh layers. Total trainable parameters: 6,238, below the 2-million limit.
- I collect 10,000 teacher examples, then use 120 epochs of behavior cloning. The loss is mean squared error between the actor's mean action and the teacher action. Adam learning rate is 0.003. This gives a useful starting policy.
- I then run real PPO training for $5 \times 8192 = 40960$ environment steps. Opponents are chosen 50/50 between Rusher and Sniper at episode reset. Spawns remain random.
- PPO uses four environments, rollout length 512 per environment, batch size 256, five optimizer epochs, learning rate 10^{-6}, $\gamma = 0.995$, GAE $\lambda = 0.95$, clip range 0.1, entropy coefficient 0 and initial log standard deviation -3.5.
- Seed is 42. The small learning rate and small action noise help preserve aiming and dodging from imitation. The final exported model is the last 40,960-step checkpoint, not the validation-best file.

Q4. Reward, evidence and failed ideas (continued)

Final reward: the original defense profile

Event or progress	Reward
Final kill / being killed	$+20$ / -20
Enemy ejection / own ejection	$+3$ / -3
Pickup / each step	$+0.1$ / -0.002
Respawn / asteroid hit	0 / 0
Distance progress	$c(d_{\text{previous}} - d_{\text{now}})$
Coefficient c	own pilot: -0.5; enemy pilot: 2; otherwise: 0.5

Distance is divided by 600. There is no velocity bonus in the final defense profile. Respawn has no positive reward, so repeatedly losing the ship is not useful by itself.



Original reference run: validation seeds 10000–10049, 50 games per opponent. The second plot is PPO critic loss. A low critic loss alone is not evidence of winning.

Test set	Rusher	Sniper	Games per opponent
Public seeds 0–99	95%	91%	100
Holdout seeds 50000–50099	92%	87%	100
Validation seeds 10000–10099	90%	84%	100

- The public evaluator gives 10/10 baseline points. Draws count as failures. The public set was used during development, so I do not call it an untouched test set. The validation result of 84% vs. Sniper also shows that passing is not guaranteed for every seed set.
- I tested hiding behind the central planet, peeking to shoot, and fast same-direction escape. The fast version increased Sniper-game ship speed from 128.5 to 166.2 pixels/s, but on seeds 65000–65099 it scored only 72% vs. Rusher and 80% vs. Sniper. It was not selected.
- A matched fast-controller test without velocity reward was already equally fast. Thus, the new navigation was the main speed improvement; extra speed reward did not show an extra speed benefit. Faster movement did not mean a better final policy.
- My fresh bare-command retraining gives exactly the same policy tensors as the passed reference. ZIP bytes can differ because saved source and metadata differ. Fresh stock evaluation also gives 95% and 91% on the actual hand-in copy.

Q5. PPO vs. A2C: hypothesis and controlled experiment (9 pts.)

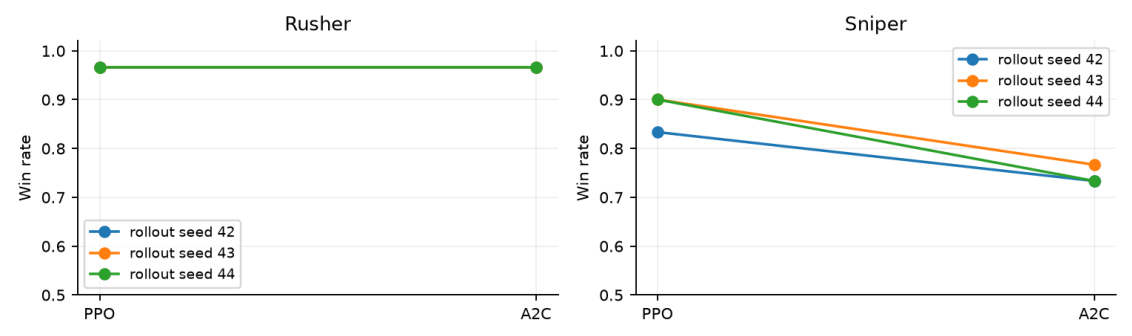
Hypothesis

- A good imitation policy can lose useful behavior after noisy updates. My hypothesis is that PPO's clipped updates and reuse of each rollout will preserve the defensive policy better than A2C, especially against Sniper.
- This experiment tests short-budget fine-tuning from the same BC policy. It does not test which algorithm learns this game best from random initialization.

What is kept the same

- Each pair starts with exactly the same actor and critic weights from the 10k-example BC checkpoint. Tensor equality is checked before training. Both use the same extractor, action noise, defense controller and original reward.
- Both get 8192 new environment steps, four environments, rollout length 512, learning rate 10^{-6}, $\gamma = 0.995$, GAE $\lambda = 0.95$, entropy coefficient 0, and normalized advantages. Both use Adam.
- I repeat with rollout seeds 42, 43 and 44. Each algorithm pair uses matching environment seeds. These runs share the BC checkpoint, so they are three rollout-seed repeats, not three fully independent BC training runs.
- Evaluation uses the same held-out seeds 72000–72029: 30 games against each bot per run. There is no model selection on this evaluation set. PPO keeps its five epochs, batch size 256 and clip range 0.1; A2C makes one update per rollout.

Rollout seed	Algorithm	Rusher win rate	Sniper win rate
42	PPO	96.7%	83.3%
42	A2C	96.7%	73.3%
43	PPO	96.7%	90.0%
43	A2C	96.7%	76.7%
44	PPO	96.7%	90.0%
44	A2C	96.7%	73.3%



Each line connects a matched pair with the same rollout seed. Games in the same seed set are not independent new test sets across the three runs.

Q5. Interpretation and limits (continued)

- Mean Rusher win rates are 96.7% for PPO and 96.7% for A2C. Mean Sniper win rates are 87.8% and 74.4%. This is a descriptive average across the three rollout seeds.
- PPO has a higher Sniper win rate in each matched pair. This supports the hypothesis in this small warm-start experiment, but does not prove the mechanism.
- PPO clips policy-ratio updates and reuses each rollout for five epochs. A2C updates once without clipping. This may help PPO keep useful aiming behavior.
- This is an equal-environment-step comparison, not equal optimizer work or wall time: PPO uses more minibatch updates. I cannot isolate clipping from sample reuse with this one experiment. A2C might need a different learning rate or more training.
- Thirty games per opponent is a small sample. All pairs share BC data and test seeds. This does not establish a universal winner.
- The full final model uses 40,960 PPO steps. These 8192-step experiment models are only for Q5 and do not replace the submitted model.

Reproduce and evaluate

- In the original Astro Party game folder, run `python train_astro_party.py`. It starts from teacher data and random network initialization, with no old checkpoint required. Output: `models/model_astro_party.zip`.
- In the original 2048 game folder, run `python train_2048.py`. The submitted configuration is the `ddqn_weight1000` recipe with standard output name `models/model_2048.zip`. Its original seed was not fixed, so this is approximate recipe reproduction.
- Evaluate with the original scripts: `python eval_astro_party.py --model models/model_astro_party.zip` and `python eval_2048.py --model models/model_2048.zip`.
- Both scripts support the flat hand-in and original `envs/` layout. Run them inside the original game folders, with the TA's game files and bots present.
- Verified runtime: Python 3.11, CPU PyTorch 2.9.1 and SB3 2.9.0. The handout suggests Python 3.10; exact cross-version tensor reproduction has not been tested.

Logs, references and assistance

- Astro Party scalar training logs are on W&B, project `astro-party-hw3`. Reference run: `6o6gk5el`; clean bare-command reproduction: `ghme0u0w`. Training records reward, PPO losses and validation wins. Q5 run: `znamd5vr`; its scalar record is synced to the same W&B project (scalar metrics and settings only).
- Local TensorBoard logs, MC returns and analysis scripts are kept outside the seven-file hand-in folder.
- Sources: assignment README, Assignment 3 slides (report p.49, policy p.51, submission p.53), provided report template, original game and bot code, SB3 DQN/PPO/A2C implementations, and the Double DQN paper by van Hasselt et al. (2015), https://arxiv.org/abs/1509.06461.
- AI assistance was used for code, experiments and drafting. Numerical results come from saved logs. The text needs personal review under the course's report policy.